

Scheduling on Unifrom Parallel Machines

given n jobs each:

p_i : processing time

r_i : release time

d_i : deadline ($d_i - r_i \geq p_i$)

Schedule on parrallel machines M with pre-emption (pausing and switching between machines) but jobs only can be processed on one machine at a time and each machine can only work on one job.

We are also able to convert this into a matrix of $|int| \times |j|$ where int is the intervals and j is the jobs. This represents the effort to each interval. We fill the matrix with the amount of processing time a job has at each interval, thus the columns should add up to the total required processing time for that job, and the rows should add up to the total amount of processing time that occured in that interval.

Reducing Scheduling Problem to Max Flow

Start with source node s and end node t . s has arcs to each of the intervals, with an upperbound of $|M| * |int|$. Then each interval node points to the jobs that are available in the interval, this has a upper bound of $|int|$. Each of the job nodes then points to t with upperbound of total processing time for that node. Thus, if the flow is not equal to the sum of all processing times then the schedule is not feasible. Otherwise, we can look at the flows on the arcs from the intervals to the jobs to determine how much processing time for a job must occur in a given interval.

Once we have the processing time in a given interval. The algorithm basically schedules jobs greedily on the same machine until it runs out of space, once it does then start over on new machine. We know there can't be any overlap because this would imply that a given job requires more processing time than the interval.